

Fase 1 · Definición del problema, entorno reproducible y estructura del proyecto

MCDIA500 · Programación para la Ciencia de Datos · Grupo 6 Integrantes: *Fernanda Ovalle Román · Sebastián Cajales Cid · César Lorca Bacián* Repositorio: <https://github.com/Feroroman/proyecto-grupo6-mcdi500>

Este notebook documenta la Fase 1 del proyecto transversal. No analiza datos: deja las condiciones para que el análisis de la Fase 2 sea reproducible, trazable y compartido. La lógica técnica vive en la clase `ProyectoF1` (`src/proyecto.py`), probada en `tests/test_proyecto.py`; aquí se orquesta y se documenta.

Antes de entregar: **Kernel** → **Restart Kernel and Run All Cells**. Se ejecuta con el kernel *Python (mcdi500)*; las rutas se resuelven desde la raíz del repositorio.

1. Contexto problematizador y pregunta analizable

Situación. En Chile la duración real de las carreras universitarias suele superar la duración formal, con costos para estudiantes e instituciones. El Mineduc publica cada año los registros de titulados, pero no está claro qué características de la carrera y de la institución acompañan una titulación más larga.

Problema. No se conoce qué factores observables se asocian a una mayor duración hasta el título.

Pregunta analizable. ¿Qué factores observables —tipo de universidad (CRUCH/privada), jornada, modalidad, área de conocimiento, región de la sede, sexo y rango etario— se asocian a un mayor número de semestres hasta la obtención del título (`dur_total_carr`) entre los titulados de pregrado universitario de 2025?

Objetivo general. Caracterizar la duración de la titulación en el pregrado universitario chileno (cohorte 2025) e identificar los factores observables asociados a una mayor duración, mediante un flujo de trabajo reproducible en Python.

Objetivos específicos. 1. Obtener y verificar el conjunto de datos de titulados 2025 del Mineduc, documentando origen, variables y problemas de calidad (F1). 2. Construir un pipeline de limpieza y transformación que justifique cada decisión con cifras (F2). 3. Implementar funciones y clases reutilizables para calcular y comparar la duración por grupos (F3). 4. Comunicar los hallazgos con visualizaciones e informe técnico (F4).

Alcance. Solo pregrado en universidades (quedan fuera IP, CFT, postítulos y posgrados). Análisis descriptivo y de asociación; no se construyen modelos predictivos.

Por qué es analizable. Los datos existen (`dur_total_carr` como variable de respuesta y los factores como columnas del mismo registro), los conceptos se miden con esas variables y el alcance está acotado.

In [1]:

```
# Ubicar la raíz del repositorio (funciona desde notebooks/F1/ o desde la raíz) y
cargar el módulo del proyecto

from pathlib import Path

import os, sys

RAIZ = Path.cwd()

while not (RAIZ / "README.md").exists() and RAIZ.parent != RAIZ:
```

```

RAIZ = RAIZ.parent
os.chdir(RAIZ); sys.path.insert(0, str(RAIZ))

from src.proyecto import ProyectoF1 # clase que encapsula configuración y
operaciones de la Fase 1

# Toda la configuración del proyecto en un solo objeto: el README, las rutas y el
filtro se derivan de aquí
proyecto = ProyectoF1(
    grupo="Grupo 6",
    integrantes=["Fernanda Ovalle Román", "Sebastián Cajales Cid", "César Lorca
Bacián"],
    repositorio="https://github.com/Feroroman/proyecto-grupo6-mcdi500",
    pregunta=("¿Qué factores observables (tipo de universidad, jornada, modalidad,
área de conocimiento, "
        "región de la sede, sexo y rango etario) se asocian a un mayor número
de semestres hasta la "
        "obtención del título entre los titulados de pregrado universitario de
2025?"),
    fuente_nombre="Titulados de Educación Superior 2025 - Datos Abiertos Mineduc",
    fuente_url="https://datosabiertos.mineduc.cl/",
    fuente_licencia=("Datos públicos del Estado de Chile, de acceso libre y
gratuito, publicados por el Centro de Estudios "
        "del Mineduc; el portal no declara una licencia Creative
Commons específica y se reutilizan con "
        "atribución a la fuente conforme a la Ley 20.285 sobre acceso a
la información pública"),
    archivo_original=Path("data/raw/20260817_Titulados_Ed_Superior_2025_WEB.csv"),
    archivo_subconjunto=Path("data/processed/titulados_2025_pregrado_univ.csv"),
    filtro={"nivel_global": "Pregrado", "tipo_inst_1": "Universidades"},
    variable_respuesta="dur_total_carr",
    factores=["tipo_inst_2", "jornada", "modalidad", "area_conocimiento",
"region_sede", "gen_alu", "rango_edad"],
)
print("Raíz del proyecto:", RAIZ)
print("Pregunta:", proyecto.pregunta)

```

Out:

```

Raíz del proyecto:
/tmp/claude-0/-home-claude/7d5d987b-d69f-5ff6-b6ac-07de88f524e4/scratchpad/repo

Pregunta: ¿Qué factores observables (tipo de universidad, jornada, modalidad, área de
conocimiento, región de la sede, sexo y rango etario) se asocian a un mayor número de
semestres hasta la obtención del título entre los titulados de pregrado universitario
de 2025?

```

2. Entorno reproducible

Reproducibilidad = mismos datos + mismo código + mismo entorno. El entorno virtual `.venv` aísla las librerías, `requirements.txt` declara sus versiones y el kernel del notebook debe ser el del proyecto (**Python (mcdi500)**). `verificar_entorno()` lanza un error explícito si no es así.

In [2]:

```

info = proyecto.verificar_entorno(exigir_venv=True) # falla si el kernel no es el
de .venv

for k, v in info.items():

```

```
| print(f"{k:<11}: {v}")
```

| Out:

```
| python      : 3.11.15
| interprete  : /tmp/claude-0/.venv/bin/python3
| pandas     : 3.0.2
```

3. Estructura del repositorio

Cada carpeta corresponde a una etapa del ciclo de vida del dato. El dato crudo (data/raw/) nunca se modifica; cada transformación produce una versión nueva en data/processed/.

| In [3]:

```
| estado = proyecto.verificar_estructura(RAIZ)
| for ruta, existe in estado.items():
|     print(f"'OK ' if existe else 'FALTA' {ruta:<18}
|     {proyecto.estructura[ruta]}")
| assert all(estados.values()), "Faltan elementos de la estructura del repositorio"
```

| Out:

```
| OK    data/raw/          dato crudo descargado del portal; no se modifica ni se
| versiona
| OK    data/processed/    derivados regenerables: subconjunto (F1) y versiones limpias
| (F2)
| OK    notebooks/        F1/F1_Definicion.ipynb · F2/F2_Pipeline.ipynb
| OK    src/              módulos: proyecto, exploracion, limpieza, transformacion,
| escalado, validacion, bitacora
| OK    tests/            pruebas de las funciones (caso normal, límite, excepción)
| OK    docs/             validación del dataset, bitácora, verificación, figuras,
| evidencias
| OK    requirements.txt   versiones declaradas del entorno
| OK    .gitignore        qué NO se versiona: .venv/, todos los CSV, salidas de
| notebooks, .idea/
| OK    README.md         qué es el proyecto y cómo ejecutarlo desde cero
```

4. Conjunto de datos: origen, verificación y subconjunto

La base nacional completa (175,6 MB) es el **dato crudo**: se descarga del portal, se deja en data/raw/ y no se modifica ni se versiona (supera el límite de 100 MB de GitHub). El proyecto trabaja sobre un **subconjunto derivado**: titulados de **pregrado en universidades**. El filtro es una decisión técnica: mantiene una población homogénea para la pregunta (la duración de un magíster o de una carrera técnica no es comparable con la de un pregrado universitario).

El subconjunto es un derivado: se escribe en data/processed/ y tampoco se versiona; cualquier integrante lo regenera con generar_subconjunto(), que además verifica que quede bajo 100 MB.

| In [4]:

```
| import pandas as pd
| df, mensaje = proyecto.generar_subconjunto(limite_mb=100)
| print(mensaje)
| print(f"Tamaño en disco: {proyecto.archivo_subconjunto.stat().st_size /
| 1_048_576:.1f} MB (límite GitHub: 100 MB)")
```

| Out:

```
| Base completa: 328,998 filas → subconjunto: 105,063 filas guardado en
| data/processed/titulados_2025_pregrado_univ.csv
```

Tamaño en disco: 55.8 MB (límite GitHub: 100 MB)

In [5]:

```
# Verificación con el validador del curso (src/validar_dataset.py) → docs/validacion_subconjunto.md

import subprocess

res = subprocess.run([sys.executable, "src/validar_dataset.py",
str(proyecto.archivo_subconjunto), "--sep", proyecto.separador,
                        "--informe", "docs/validacion_subconjunto.md"],
capture_output=True, text=True)

assert res.returncode == 0, res.stderr      # si el validador falla, el notebook
debe fallar

print(res.stdout)

print(Path("docs/validacion_subconjunto.md").read_text(encoding="utf-8").split("##
Alertas")[-1])
```

Out:

```
Informe guardado en docs/validacion_subconjunto.md
Alertas: 4
```

- ⚠ Hay 3 filas duplicadas.
- ⚠ `cat\_perodo` es constante (un solo valor).
- ⚠ `tipo\_inst\_1` es constante (un solo valor).
- ⚠ `nivel\_global` es constante (un solo valor).

Roles analíticos de las variables

Cada rol exige un tratamiento distinto en la Fase 2. Declararlos aquí es lo que después justifica cada decisión.

In [6]:

```
ROLES = {
    "identificador": ["mrun"],
    "numerica (semestres)": ["dur_estudio_carr", "dur_proceso_tit",
"dur_total_carr"],
    "numerica discreta": ["anio_ing_carr_ori", "anio_ing_carr_act", "version"],
    "binaria": ["gen_alu"],
    "nominal": ["tipo_inst_2", "jornada", "modalidad",
"area_conocimiento", "region_sede", "tipo_plan_carr"],
    "ordinal": ["rango_edad", "nivel_carrera_1"],
    "fecha": ["fecha_obtencion_titulo", "fec_nac_alu"],
    "alta_cardinalidad": ["nomb_carrera", "nomb_inst", "nomb_sede",
"comuna_sede"],
    "constante_descartar": ["cat_perodo", "tipo_inst_1", "nivel_global"],
}

faltantes = df.isna().mean().mul(100).round(1)

for rol, cols in ROLES.items():
    print(f"\n{rol}:")
    for c in cols:
        print(f"    {c:<24} nulos {faltantes[c]:>5}%    únicos {df[c].nunique():>7,}")
```

Out:

identificador:					
	mrún	nulos	0.0%	únicos	104,595
numerica (semestres):					
	dur_estudio_carr	nulos	0.0%	únicos	15
	dur_proceso_tit	nulos	0.0%	únicos	7
	dur_total_carr	nulos	0.0%	únicos	16
numerica_discreta:					
	anio_ing_carr_ori	nulos	0.0%	únicos	45
	anio_ing_carr_act	nulos	0.0%	únicos	42
	version	nulos	0.3%	únicos	8
binaria:					
	gen_alu	nulos	0.0%	únicos	2
nominal:					
	tipo_inst_2	nulos	0.0%	únicos	2
	jornada	nulos	0.0%	únicos	5
	modalidad	nulos	0.0%	únicos	3
	area_conocimiento	nulos	0.0%	únicos	10
	region_sede	nulos	0.0%	únicos	16
	tipo_plan_carr	nulos	0.0%	únicos	3
ordinal:					
	rango_edad	nulos	0.0%	únicos	6
	nivel_carrera_1	nulos	0.0%	únicos	4
fecha:					
	fecha_obtencion_titulo	nulos	0.0%	únicos	355
	fec_nac_alu	nulos	0.0%	únicos	585
alta_cardinalidad:					
	nomb_carrera	nulos	0.0%	únicos	1,096
	nomb_inst	nulos	0.0%	únicos	55
	nomb_sede	nulos	0.0%	únicos	109
	comuna_sede	nulos	0.0%	únicos	56
constante_descartar:					
	cat_periodo	nulos	0.0%	únicos	1
	tipo_inst_1	nulos	0.0%	únicos	1
	nivel_global	nulos	0.0%	únicos	1

Problemas de calidad detectados (insumo de la Fase 2)

Se miden aquí; se resuelven y justifican en notebooks/F2/F2_Pipeline.ipynb.

In [7]:

```
cod_1900 = int((df["anio_ing_carr_ori"] == 1900).sum())
print(f"anio_ing_carr_ori = 1900 (código «sin información»): {cod_1900:,} filas
      ({100*cod_1900/len(df):.1f} %)")
print(f"Filas duplicadas: {df.duplicated().sum():,}")
print(f"mr_run nulo: {df['mr_run'].isna().sum():,}")
print(f"nombre_titulo nulo: {100*df['nombre_titulo'].isna().mean():.1f} % .
      nombre_grado nulo: {100*df['nombre_grado'].isna().mean():.1f} %")
print(f"Columnas constantes: {[c for c in df.columns if df[c].nunique() == 1]}")
print("\nDistribución de la variable de respuesta (semestres):")
print(df[proyecto.variable_respuesta].describe().round(2).to_string())
```

Out:

```
anio_ing_carr_ori = 1900 (código «sin información»): 13,396 filas (12.8 %)
Filas duplicadas: 3
mr_run nulo: 7
nombre_titulo nulo: 2.9 % . nombre_grado nulo: 8.0 %
Columnas constantes: ['cat_periodo', 'tipo_inst_1', 'nivel_global']

Distribución de la variable de respuesta (semestres):
count    105063.00
mean         9.07
std         2.44
min          1.00
25%          8.00
50%         10.00
75%         10.00
max         24.00
```

5. Control de versiones y trabajo colaborativo

- **Git** registra el historial local con autoría; **GitHub** aloja el repositorio remoto compartido por los tres integrantes.
- Convención de commits tipo: descripción con los prefijos docs, data, feat, fix, test.
- Una rama por integrante y fase, integrada a main mediante pull request; conflictos resueltos con git pull, nunca con --force.
- nbstripout activo en los tres equipos: limpia las salidas del notebook antes de cada commit.
- Ningún CSV se versiona (ver .gitignore): el repositorio contiene el código que reproduce los datos, no los datos.

6. README generado desde la configuración

El README se compone desde el objeto proyecto, así la documentación no se desactualiza respecto del código.

In [8]:

```
readme = proyecto.generar_readme()
Path("README.md").write_text(readme, encoding="utf-8")
assert "TU_USUARIO" not in readme and "completar" not in readme.lower() # sin
marcadores pendientes
```

```

| print(readme[:1200], "\n[...]")
Out:

| # Duración de la titulación en el pregrado universitario chileno (2025)

| **MCDI500 · Programación para la Ciencia de Datos** · Grupo 6

| ## Integrantes
| - Fernanda Ovalle Román
| - Sebastián Cajales Cid
| - César Lorca Bacián

| ## Problemática y pregunta
| La duración real de las carreras universitarias en Chile suele superar la formal. Este
| proyecto analiza qué
| factores observables acompañan una titulación más larga.

| **Pregunta analizable:** ¿Qué factores observables (tipo de universidad, jornada,
| modalidad, área de conocimiento, región de la sede, sexo y rango etario) se asocian a
| un mayor número de semestres hasta la obtención del título entre los titulados de
| pregrado universitario de 2025?

| **Alcance:** solo pregrado en universidades; análisis descriptivo y de asociación, sin
| modelos predictivos.

| ## Conjunto de datos
| - Fuente: Titulados de Educación Superior 2025 – Datos Abiertos Mineduc –
| https://datosabiertos.mineduc.cl/
| - Licencia: Datos públicos del Estado de Chile, de acceso libre y gratuito, publicados
| por el Centro de Estudios del Mineduc; el portal no declara una licencia Creative
| Commons específica y se reutilizan con atribución a la fuente conforme a la Ley 20.285
| sobre acc
| [...]

```

7. Vinculación con el mapa conceptual (Formativa 1)

Elemento del mapa	Estado
Problema → pregunta analizable → dataset	Implementado (secciones 1 y 4)
Python + entorno virtual + requirements.txt	Implementado (sección 2, verificar_entorno)
Estructura del repositorio, README, .gitignore	Implementado (secciones 3 y 6)
Git / GitHub, convención de commits	Implementado (sección 5; historial en GitHub)
Funciones y clases reutilizables (src/) con pruebas	Implementado: ProyectoF1 + tests/test_proyecto.py
Ciclo de vida: exploración → transformación → data/processed	Implementado en

Elemento del mapa	Estado
	notebooks/F2/F2_Pipeline.ipynb
Bitácora de decisiones	Implementado en la Fase 2 (docs/bitacora.md)
Análisis y comunicación, informe final	Proyectado: Fases 3 y 4